Team 6

Angelo Torres 862477027, Tommy Haley

EE 128 Password-based Door Lock Project with Timer and Interrupt extension

Section 21

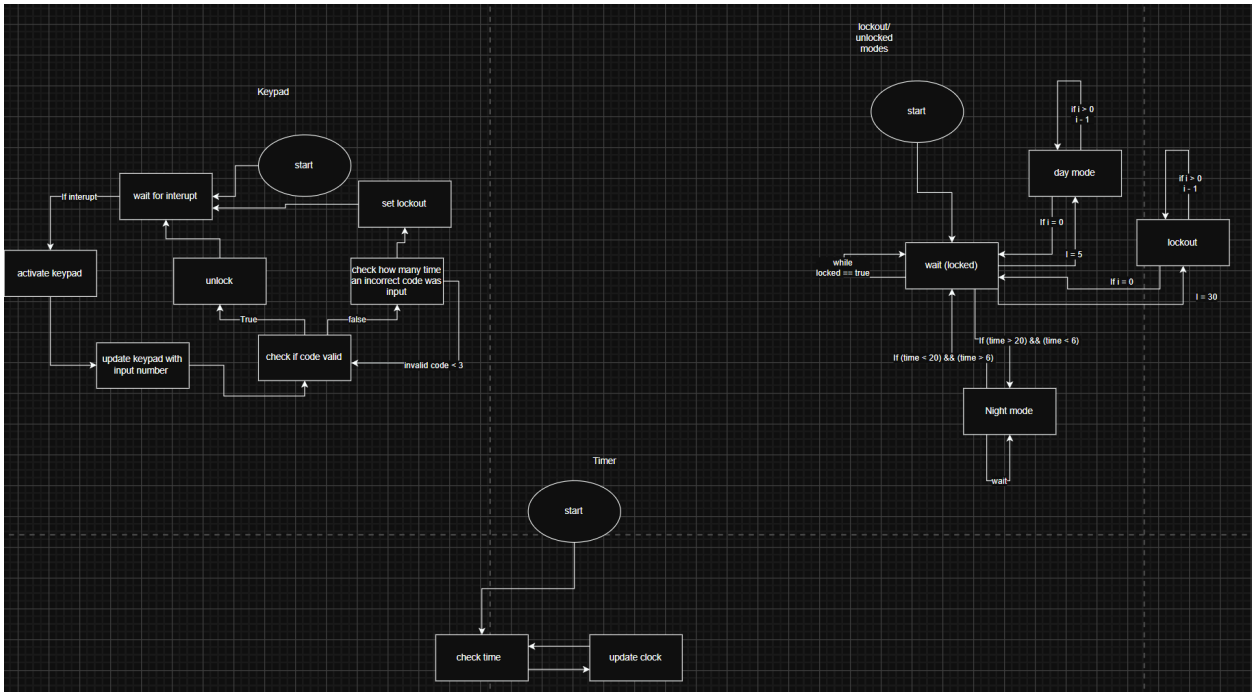
Johnson Zhang

Abstract

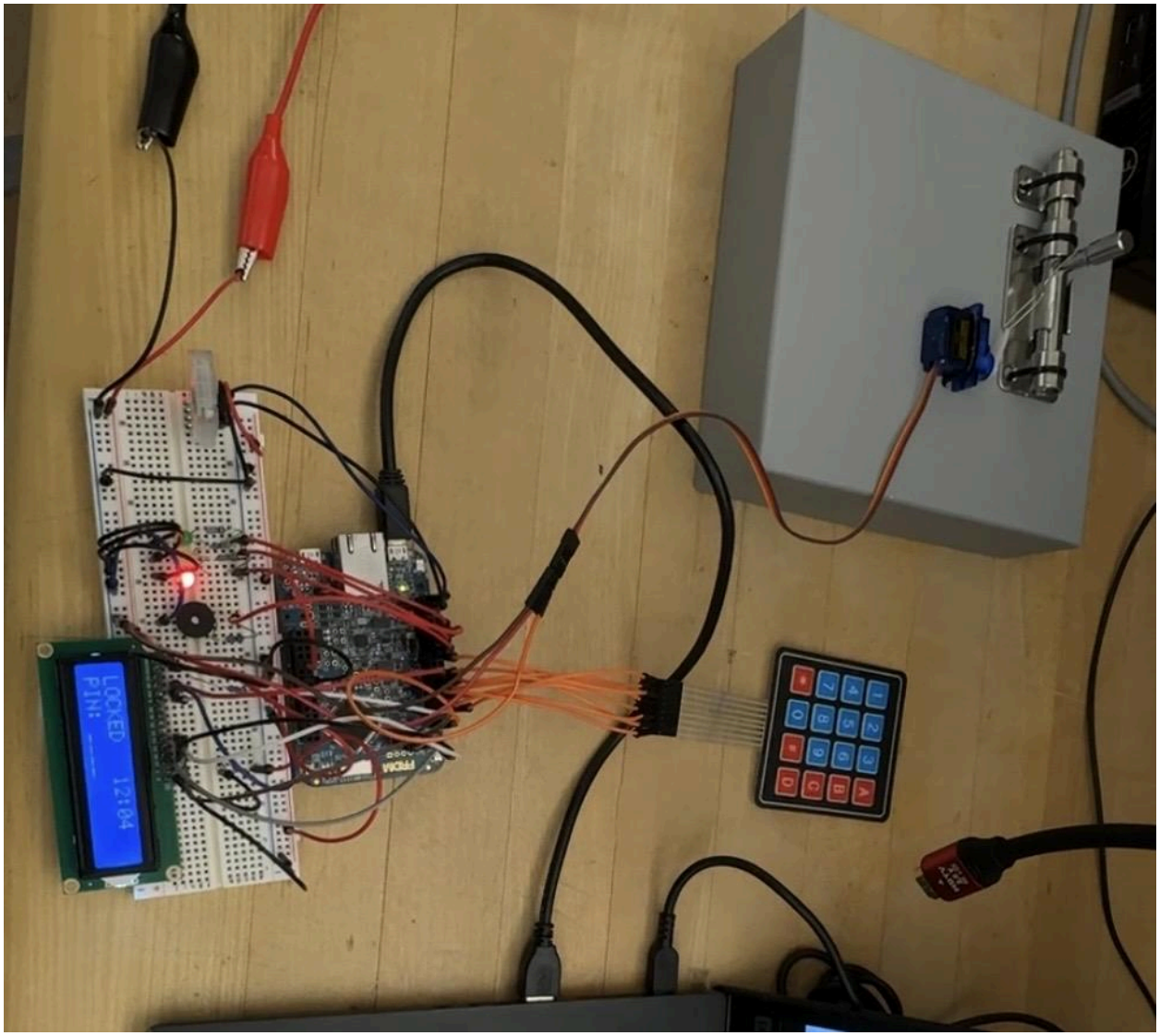
The main goal of this project was to create a reliable smart door lock system that works based on events, bridging the gap between theory and practical applications. We set it up using four different timers: a PITO hardware interval timer that sends out a heartbeat every 10 milliseconds to handle the auto-lock (every 5 or 2 seconds) and lockout (30 seconds) features; an FTM2 module in PWM mode to generate a nice 50 Hz signal for controlling the angles of the servo; an FTM3 module to provide audio feedback with digital waveforms; and a Software Real-Time Clock (RTC) that compiles those PIT ticks to keep track of time for the "Night Mode" features. By using a "Vector Table" approach with three types of interrupts, we got rid of inefficient polling: there are GPIO interrupts on Ports B and C that only wake up the CPU when someone presses the keypad, periodic timer interrupts for managing background tasks, and UART4 receive interrupts that handle Bluetooth commands as soon as they arrive.

Experiment System Specification

Flowchart diagram


Keypad
start
wait for interupt
If interupt
activate keypad
unlock
set lockout
check how many time an incorrect code was input
True
false
update keypad with input number
check if code valid
invalid code < 3
lockout/ unlocked modes
start
if i > 0 i - 1
day mode
if i > 0 i - 1
lockout
if i = 0
i = 5
while locked == true
wait (locked)
if i = 0
i = 30
If (time > 20) && (time < 6)
If (time < 20) && (time > 6)
Night mode
wait
Timer
start
check time
update clock

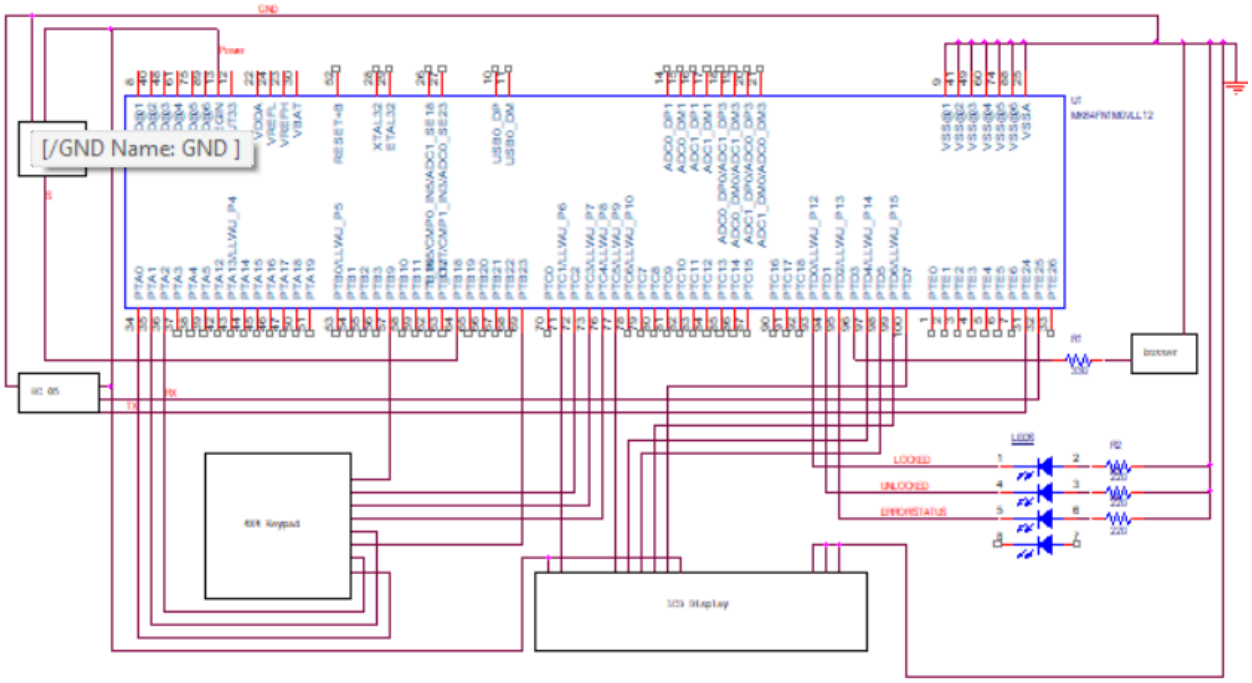
Photos



Video Demo

Hardware Design

Schematic



Software Design

High-Level Description:

The firmware creates an efficient, event-driven system for the MK64F12 microcontroller using a Finite State Machine (FSM) to handle different states like LOCKED, UNLOCKED, and LOCKOUT based on various signals. This setup avoids delays by using a foreground-background model where the main loop continuously checks "dirty" flags set off by three types of interrupts. A periodic PIT0 interrupt acts like a heartbeat every 10ms, helping with background tasks like auto-lock and lockout countdowns, and also compiles ticks for a software Real-Time Clock (RTC) that enables a time-sensitive "Night Mode." For automation, it directly changes registers: FTM2 is configured for 50Hz PWM to adjust servo angles by tweaking duty cycles, FTM3 creates sound feedback by calculating modulo values for different frequencies, and UART4 receives Bluetooth commands instantly into a global buffer, allowing remote control without disturbing the processor. Finally, security is maintained through GPIO interrupts on Ports B and C, which activate the CPU only when the keypad is engaged, so it can scan without blocking, keeping the system responsive to user inputs and internal timing tasks.

Code:

```
#include "MK64F12.h"
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>

#define LED_ACTIVE_HIGH 1

#if LED_ACTIVE_HIGH
  #define LED_ON(bit)  (GPIOD_PSOR = (1u << bit))
  #define LED_OFF(bit) (GPIOD_PCOR = (1u << bit))
#else
  #define LED_ON(bit)  (GPIOD_PCOR = (1u << bit))
  #define LED_OFF(bit) (GPIOD_PSOR = (1u << bit))
#endif

#define PIN_LEN         4
#define PIT0_TICK_MS    10U

// Auto-lock times
#define AUTOLOCK_TICKS_DAY   (5000U / PIT0_TICK_MS)
#define AUTOLOCK_TICKS_NIGHT (2000U / PIT0_TICK_MS)

// Lockout time
#define LOCKOUT_TICKS        (30000U / PIT0_TICK_MS)
#define MAX_FAILED_ATTEMPTS  3

/* Hardware Constants */
#define FTM2_MOD_VALUE      3276U
#define SERVO_LOCKED_CNV    164U
#define SERVO_UNLOCKED_CNV  328U
#define FTM3_CLK_FREQ       1310720

typedef enum {
    STATE_LOCKED = 0,
    STATE_UNLOCKED,
    STATE_PERM_UNLOCKED,
    STATE_CHANGE_OLD_PIN,
    STATE_CHANGE_NEW_PIN,
    STATE_LOCKOUT,
    STATE_BLUETOOTH_MODE
} LockState;

static char stored_pin[PIN_LEN] = { '1','2','3','4' };
```

```
static char input_buf[PIN_LEN];
static uint8_t input_len = 0;
static LockState lock_state = STATE_LOCKED;
static uint8_t failed_attempts = 0;

static bool cfg_night_mode_enabled = false;

// Timer Globals
volatile int32_t autolock_ticks = -1;
volatile int32_t lockout_ticks = -1;
static int32_t last_display_seconds = -1;

// Clock Globals
volatile uint8_t rtc_hour = 12;
volatile uint8_t rtc_min  = 00;
volatile uint8_t rtc_sec  = 00;
volatile bool is_night_time = false;
static uint8_t pit_subsecond_count = 0;


volatile char g_key_pressed = 0;
static uint8_t lcd_dirty = 1;

// Bluetooth Globals
volatile char g_bt_command = 0;
static bool bt_locked_state = false;
static void delay_ms(uint32_t ms) {
    volatile uint32_t i;
    while (ms--) for (i = 0; i < 21000; i++) __asm("nop");
}

static void GPIO_Init(void) {
    SIM_SCGC5 |= SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK |
SIM_SCGC5_PORTC_MASK | SIM_SCGC5_PORTD_MASK | SIM_SCGC5_PORTE_MASK;

    // LEDs
    PORTD_PCR0 = PORT_PCR_MUX(1); PORTD_PCR1 = PORT_PCR_MUX(1);
PORTD_PCR2 = PORT_PCR_MUX(1);
    GPIOD_PDDR |= (1u<<0) | (1u<<1) | (1u<<2);
    LED_OFF(0); LED_OFF(1); LED_OFF(2);

    // LCD
    PORTC_PCR5 = PORT_PCR_MUX(1); PORTC_PCR1 = PORT_PCR_MUX(1);
    GPIOC_PDDR |= (1u<<5) | (1u<<1);
```

```
    PORTD_PCR4 = PORT_PCR_MUX(1); PORTD_PCR5 = PORT_PCR_MUX(1);
PORTD_PCR6 = PORT_PCR_MUX(1); PORTD_PCR7 = PORT_PCR_MUX(1);
    GPIOD_PDDR |= (1u<<4) | (1u<<5) | (1u<<6) | (1u<<7);

    // Keypad Rows
    PORTB_PCR9 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    PORTC_PCR2 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    PORTC_PCR3 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    GPIOB_PDDR &= ~(1u<<9); GPIOC_PDDR &= ~((1u<<2) | (1u<<3) | (1u<<4));
    NVIC_EnableIRQ(PORTB_IRQn); NVIC_EnableIRQ(PORTC_IRQn);

    // Keypad Cols
    PORTA_PCR1=PORT_PCR_MUX(1); PORTB_PCR23=PORT_PCR_MUX(1);
PORTA_PCR2=PORT_PCR_MUX(1); PORTC_PCR0=PORT_PCR_MUX(1);
    GPIOA_PDDR |= (1u<<1)|(1u<<2); GPIOB_PDDR |= (1u<<23); GPIOC_PDDR |= (1u<<0);
    GPIOA_PCOR = (1u<<1)|(1u<<2); GPIOB_PCOR = (1u<<23); GPIOC_PCOR = (1u<<0);
}

// UART Bluetooth
static void UART4_Init(void) {
    SIM_SCGC1 |= SIM_SCGC1_UART4_MASK;
    PORTE_PCR24 = PORT_PCR_MUX(3); // TX
    PORTE_PCR25 = PORT_PCR_MUX(3); // RX
    UART4_C2 &= ~(UART_C2_TE_MASK | UART_C2_RE_MASK);
    uint16_t sbr = (uint16_t)(21000000 / (16 * 9600));
    UART4_BDH = (sbr >> 8) & 0x1F;
    UART4_BDL = (sbr & 0xFF);
    UART4_C2 |= UART_C2_RIE_MASK;
    NVIC_EnableIRQ(UART4_RX_TX_IRQn);
    UART4_C2 |= (UART_C2_TE_MASK | UART_C2_RE_MASK);
}

void UART4_RX_TX_IRQHandler(void) {
    if (UART4_S1 & UART_S1_RDRF_MASK) {
        char c = UART4_D;
        g_bt_command = c;
    }
}
```

```
// Outputs
static void Set_LEDs(bool locked, bool unlocked, bool error) {
    if(locked) LED_ON(0); else LED_OFF(0);
    if(unlocked) LED_ON(1); else LED_OFF(1);
    if(error) LED_ON(2); else LED_OFF(2);
}

static void Buzzer_Init(void) {
    SIM_SCGC3 |= SIM_SCGC3_FTM3_MASK; SIM_SCGC5 |= SIM_SCGC5_PORTD_MASK;
    PORTD_PCR3 = PORT_PCR_MUX(4); FTM3_MODE |= FTM_MODE_WPDIS_MASK;
    FTM3_SC=0; FTM3_CNT=0; FTM3_MOD=0; FTM3_C3SC = 0x28; FTM3_C3V=0;
    FTM3_SC = 0x0C;
}

static void Play_Tone(uint16_t freq, uint32_t ms) {
    if (cfg_night_mode_enabled && is_night_time) return;
    if (freq == 0) FTM3_C3V = 0;
    else { uint16_t mod = FTM3_CLK_FREQ / freq; FTM3_MOD = mod; FTM3_C3V = mod/2;
FTM3_CNT = 0; }
    delay_ms(ms); FTM3_C3V = 0; FTM3_MOD = 0;
}


void Jingle_Unlock() { Play_Tone(523,100); delay_ms(50); Play_Tone(659,100); delay_ms(50);
Play_Tone(784,200); }
void Jingle_Error()  { Play_Tone(150,300); delay_ms(100); Play_Tone(150,300); }
void Jingle_Beep()   { Play_Tone(2000,50); }
void Jingle_Lock()   { Play_Tone(784,100); delay_ms(50); Play_Tone(523,200); }

static void Servo_Init(void) {
    SIM_SCGC6|=SIM_SCGC6_FTM2_MASK; FTM2_MODE|=FTM_MODE_WPDIS_MASK;
    FTM2_MOD=FTM2_MOD_VALUE; FTM2_C0SC=0x28;
PORTB_PCR18=PORT_PCR_MUX(3);
    FTM2_SC=0x0F;
}
static void Servo_Move(bool open) { FTM2_C0V = open ? SERVO_UNLOCKED_CNV :
SERVO_LOCKED_CNV; }

static void LCD_SetRS(uint8_t h){if(h)GPIOC_PSOR=(1u<<5);else GPIOC_PCOR=(1u<<5);}
static void LCD_Pulse(void){GPIOC_PSOR=(1u<<1);for(int
i=0;i<800;i++);GPIOC_PCOR=(1u<<1);for(int i=0;i<800;i++);}
static void LCD_Nibble(uint8_t n){GPIOD_PCOR=(0xF<<4); if(n&1)GPIOD_PSOR=(1u<<4);
if(n&2)GPIOD_PSOR=(1u<<5); if(n&4)GPIOD_PSOR=(1u<<6);
if(n&8)GPIOD_PSOR=(1u<<7);}
```

```
static void LCD_Send(uint8_t v, uint8_t d){LCD_SetRS(d); LCD_Nibble(v>>4); LCD_Pulse();
LCD_Nibble(v&0xF); LCD_Pulse(); delay_ms(1);}
static void LCD_Init(void){delay_ms(20); LCD_Send(0x33,0); LCD_Send(0x32,0);
LCD_Send(0x28,0); LCD_Send(0x0C,0); LCD_Send(0x06,0); LCD_Send(0x01,0);
delay_ms(2);}
static void LCD_Pos(uint8_t r, uint8_t c){uint8_t a=(r==0)?0:0x40; LCD_Send(0x80|(a+c),0);}
static void LCD_Print(char *s){while(*s)LCD_Send(*s++,1);}

static void Update_Display(void) {
    char l1[17], l2[17];
    char time_s[10];
    char mode_char = ' ';
    if (is_night_time) mode_char = 'N';
    if (cfg_night_mode_enabled && is_night_time) mode_char = '!';

    sprintf(time_s, "%02d:%02d%c", rtc_hour, rtc_min, mode_char);

    switch(lock_state) {
        case STATE_LOCKED:
            snprintf(l1, 17, "LOCKED   %s", time_s);
            char mask[5];
            for(int i=0; i<4; i++) mask[i] = (i<input_len)?'*':'_';
            mask[4]=0;
            snprintf(l2, 17, "PIN: %-10s", mask);
            break;

        case STATE_UNLOCKED:
            snprintf(l1, 17, "UNLOCKED %s", time_s);
            if (autolock_ticks > 0) {
                int sec = (autolock_ticks + 99)/100;
                snprintf(l2, 17, "AUTO-LOCK: %ds  ", sec);
            } else {
                snprintf(l2, 17, "A:P B:N C:P D:BT");
            }
            break;

        case STATE_PERM_UNLOCKED:
            snprintf(l1, 17, "PERM UNLOCKED   ");
            snprintf(l2, 17, "PRESS * TO LOCK ");
            break;

        case STATE_LOCKOUT:
            snprintf(l1, 17, "LOCKED OUT!     ");
            snprintf(l2, 17, "WAIT: %ds      ", (int)((lockout_ticks+99)/100));
```

```
            break;

        case STATE_CHANGE_OLD_PIN:
            snprintf(l1, 17, "OLD PIN:       ");
            char omask[5]; for(int i=0;i<4;i++) omask[i]=(i<input_len)?input_buf[i]:'_'; omask[4]=0;
            snprintf(l2, 17, "%-16s", omask);
            break;

        case STATE_CHANGE_NEW_PIN:
            snprintf(l1, 17, "NEW PIN:       ");
            char nmask[5]; for(int i=0;i<4;i++) nmask[i]=(i<input_len)?input_buf[i]:'_'; nmask[4]=0;
            snprintf(l2, 17, "%-16s", nmask);
            break;

        case STATE_BLUETOOTH_MODE:
            snprintf(l1, 17, "BLUETOOTH MODE  ");
            if (bt_locked_state) snprintf(l2, 17, "STATUS: LOCKED  ");
            else                 snprintf(l2, 17, "STATUS: OPEN    ");
            break;

        default:
            snprintf(l1, 17, "ERROR          ");
            snprintf(l2, 17, "               ");
    }
    LCD_Pos(0,0); LCD_Print(l1);
    LCD_Pos(1,0); LCD_Print(l2);
}

static void Set_State(LockState new_state) {
    lock_state = new_state;
    input_len = 0;
    lcd_dirty = 1;

    switch(lock_state) {
        case STATE_LOCKED:
            Set_LEDs(true, false, false); Servo_Move(false); autolock_ticks = -1;
            break;
        case STATE_UNLOCKED:
            Set_LEDs(false, true, false); Servo_Move(true);
            autolock_ticks = (is_night_time && cfg_night_mode_enabled) ?
AUTOLOCK_TICKS_NIGHT : AUTOLOCK_TICKS_DAY;
            break;
        case STATE_PERM_UNLOCKED:
            Set_LEDs(false, true, true); Servo_Move(true); autolock_ticks = -1;
```

```
            break;
        case STATE_LOCKOUT:
            Set_LEDs(false, false, true);
            break;
        case STATE_BLUETOOTH_MODE:
            bt_locked_state = false;
            Set_LEDs(false, true, true);
            autolock_ticks = -1;
            break;
        default:
            Set_LEDs(false, false, true);
            break;
    }
}

static void Handle_Input(char k) {
    if (lock_state == STATE_LOCKOUT) { Jingle_Error(); return; }
    Jingle_Beep();

    switch (lock_state) {
        case STATE_LOCKED:
            if(k>='0' && k<='9' && input_len<4) input_buf[input_len++] = k;
            else if(k=='*') input_len=0;
            else if(k=='#') {
                bool match = true; for(int i=0;i<4;i++) if(input_buf[i]!=stored_pin[i]) match=false;
                if(match) { failed_attempts=0; Jingle_Unlock(); Set_State(STATE_UNLOCKED); }
                else {
                    failed_attempts++; Set_LEDs(false,false,true); Jingle_Error(); delay_ms(500);
                    if(failed_attempts >= MAX_FAILED_ATTEMPTS) { lockout_ticks =
LOCKOUT_TICKS; Set_State(STATE_LOCKOUT); }
                    else Set_State(STATE_LOCKED);
                }
            }
            break;

        case STATE_UNLOCKED:
            if(k=='*') { Jingle_Lock(); Set_State(STATE_LOCKED); }
            else if(k=='A') { Set_State(STATE_PERM_UNLOCKED); Jingle_Unlock(); }
            else if(k=='B') { cfg_night_mode_enabled = !cfg_night_mode_enabled; lcd_dirty = 1;
Jingle_Unlock(); }
            else if(k=='C') { Set_State(STATE_CHANGE_OLD_PIN); }
            else if(k=='D') { Set_State(STATE_BLUETOOTH_MODE); }
            break;
```

```
        case STATE_PERM_UNLOCKED:
            if(k=='*') { Jingle_Lock(); Set_State(STATE_LOCKED); }
            break;

        case STATE_BLUETOOTH_MODE:
            if(k=='*') { Set_State(STATE_UNLOCKED); }
            break;

        case STATE_CHANGE_OLD_PIN:
            if(k>='0' && k<='9' && input_len<4) input_buf[input_len++] = k;
            else if(k=='*') Set_State(STATE_UNLOCKED);
            else if(k=='#') {
                bool match = true; for(int i=0;i<4;i++) if(input_buf[i]!=stored_pin[i]) match=false;
                if(match) { Jingle_Unlock(); Set_State(STATE_CHANGE_NEW_PIN); }
                else { Jingle_Error(); Set_State(STATE_UNLOCKED); }
            }
            break;

        case STATE_CHANGE_NEW_PIN:
            if(k>='0' && k<='9' && input_len<4) input_buf[input_len++] = k;
            else if(k=='*') Set_State(STATE_UNLOCKED);
            else if(k=='#') {
                if(input_len==4) { for(int i=0;i<4;i++) stored_pin[i]=input_buf[i]; Jingle_Unlock();
Set_State(STATE_LOCKED); }
            }
            break;

        default: break;
    }
    lcd_dirty = 1;
}

// Timer/ISR
static char ScanKey() {
    const char t[] = {'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'};
    for(int c=0;c<4;c++) {
        GPIOA_PSOR=(1u<<1)|(1u<<2); GPIOB_PSOR=(1u<<23); GPIOC_PSOR=(1u<<0);
        if(c==0)GPIOA_PCOR=(1u<<1); if(c==1)GPIOB_PCOR=(1u<<23);
if(c==2)GPIOA_PCOR=(1u<<2); if(c==3)GPIOC_PCOR=(1u<<0);
        for(int d=0;d<400;d++)__asm("nop");
        if(!(GPIOB_PDIR&(1u<<9))) return t[c]; if(!(GPIOC_PDIR&(1u<<2))) return t[4+c];
        if(!(GPIOC_PDIR&(1u<<3))) return t[8+c]; if(!(GPIOC_PDIR&(1u<<4))) return t[12+c];
    }
    return 0;
```

```
}
void PORTB_IRQHandler(void){ delay_ms(5); char k=ScanKey(); if(k)g_key_pressed=k;
PORTB_ISFR=0xFFFFFFFF; PORTC_ISFR=0xFFFFFFFF; GPIOA_PCOR=(1u<<1)|(1u<<2);
GPIOB_PCOR=(1u<<23); GPIOC_PCOR=(1u<<0); }
void PORTC_IRQHandler(void){ PORTB_IRQHandler(); }

static void PIT0_Init(void) {
   SIM->SCGC6 |= SIM_SCGC6_PIT_MASK; PIT->MCR &= ~PIT_MCR_MDIS_MASK;
PIT->MCR |= PIT_MCR_FRZ_MASK;
   PIT->CHANNEL[0].LDVAL = 210000 - 1; PIT->CHANNEL[0].TCTRL = 3;
NVIC_EnableIRQ(PIT0_IRQn);
   if (rtc_hour >= 20 || rtc_hour < 6) is_night_time = true;
}
void PIT0_IRQHandler(void) {
   PIT->CHANNEL[0].TFLG = 1;
   if (autolock_ticks > 0) autolock_ticks--;
   if (lockout_ticks > 0) lockout_ticks--;

   // Wall Clock
   pit_subsecond_count++;
   if (pit_subsecond_count >= 100) {
      pit_subsecond_count = 0;
      rtc_sec++;
      if (rtc_sec >= 60) {
         rtc_sec = 0; rtc_min++;
         if (rtc_min >= 60) {
            rtc_min = 0; rtc_hour++;
            if (rtc_hour >= 24) rtc_hour = 0;
         }
      }
      if (rtc_hour >= 20 || rtc_hour < 6) is_night_time = true;
      else is_night_time = false;
      lcd_dirty = 1;
   }
}

int main(void) {
   GPIO_Init(); LCD_Init(); Servo_Init(); Buzzer_Init(); PIT0_Init(); UART4_Init();
   Set_State(STATE_LOCKED); Jingle_Lock();

   while (1) {
      // Timers
      if (lock_state == STATE_UNLOCKED && autolock_ticks == 0) { Jingle_Lock();
Set_State(STATE_LOCKED); }
```

```
    if (lock_state == STATE_LOCKOUT && lockout_ticks == 0) { failed_attempts = 0;
Set_State(STATE_LOCKED); Jingle_Beep(); }

    // Bluetooth Logic
    if (g_bt_command != 0) {
        char c = g_bt_command;
        g_bt_command = 0;

        if (lock_state == STATE_BLUETOOTH_MODE) {
            if (c == 'u' || c == 'U') {
                Jingle_Unlock(); Servo_Move(true); Set_LEDs(false, true, true);
                bt_locked_state = false; lcd_dirty = 1;
            }
            else if (c == 'l' || c == 'L') {
                Jingle_Lock(); Servo_Move(false); Set_LEDs(true, false, true);
                bt_locked_state = true; lcd_dirty = 1;
            }
            else if (c == 't' || c == 'T') {
                if (bt_locked_state) { Jingle_Unlock(); Servo_Move(true); Set_LEDs(false, true,
true); bt_locked_state = false; }
                else { Jingle_Lock(); Servo_Move(false); Set_LEDs(true, false, true);
bt_locked_state = true; }
                lcd_dirty = 1;
            }
        }
    }

    // Keypad Input
    if (g_key_pressed != 0) {
        char k = g_key_pressed; g_key_pressed = 0;
        Handle_Input(k);
    }

    // Update Display
    int32_t current_sec = -1;
    if (autolock_ticks > 0) current_sec = autolock_ticks / 100;
    else if (lockout_ticks > 0) current_sec = lockout_ticks / 100;

    if (lcd_dirty || (current_sec != -1 && current_sec != last_display_seconds)) {
        Update_Display();
        if(current_sec != -1) last_display_seconds = current_sec;
        lcd_dirty = 0;
    }
    delay_ms(10);
```

```
  }
}
```

Conclusion

Overall, this project really showed off the perks of using a non-blocking, event-driven setup by smoothly bringing together various hardware parts while keeping the system responsive. By assigning time-sensitive tasks like generating servo PWM signals and handling audio to specific hardware modules (FTM2 and FTM3) and managing asynchronous events through the interrupt controller, the system could pull off complex tasks like updating LCD countdowns while also keeping an eye on Bluetooth inputs. The successful addition of features like the Software RTC and permanent unlock state control highlights how well hardware timers can work with precise software logic.